
Metadata-Driven Data Science

Peter Geissert MS, MPH

Cascadia R Conference June 21, 2025



OREGON EMERGENCY MEDICAL SERVICES PROGRAM
CENTER FOR HEALTH PROTECTION

Acknowledgements

- The Oregon Emergency Medical Services Program personnel and Data Team members past and present
- ODOT and the TRCC for partial financial support of this work

Roadmap

- The public sector quandary
- A very brief introduction to metadata and metadata driven data science
- Example: Pulling a 1:1 table from our database

The Public Sector Quandary

Recent trends in public health, data science, EMS, and Trauma:

- Increasing opportunities for quality assurance and process improvement initiatives
- Public health modernization has brought in new expectations for public health programs to be data driven
- Greater visibility of data science as a discipline has brought new tools, increased data literacy, and greater demand for data
- Our data systems have expanded in coverage, making the data housed there more useful

Potemkin Village Data Science



PUBLIC HEALTH DIVISION
EMS and Trauma Systems

Oregon
Health
Authority

Potemkin Village Data Science



Getting upstream

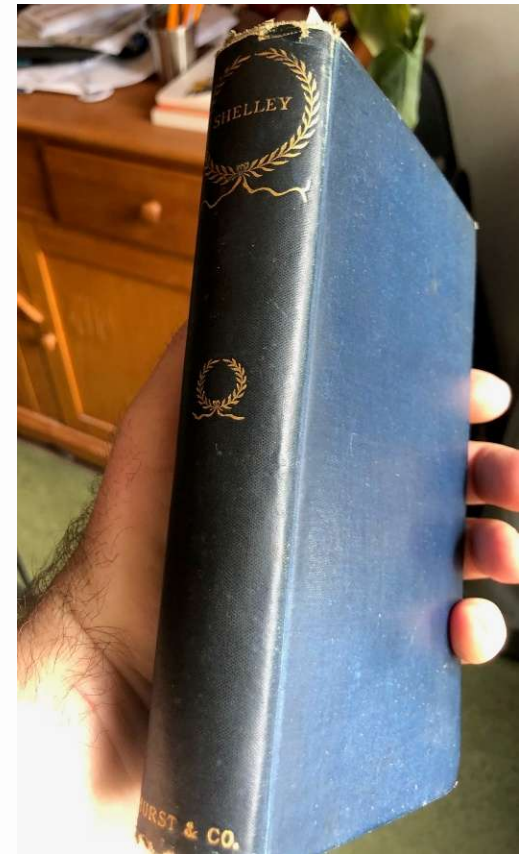
- The only way to manage this situation is to get upstream from the project
- The way to get upstream from data science projects is to work with our metadata



What is Metadata?

This volume of Shelley's poems belonged to my grandmother. If the poems are the data, here are some examples of metadata:

- Bookplate
- Publisher information
- Table of contents / Index
- Verse/Page numbers
- Book sellers' marks
- Wear on the cover, spine and pages
- Dictionary of the language
- Grammar and syntax of the language



What is Metadata?

Types of metadata

- **Structural metadata** – Information about data structures and how data are stored
- **Descriptive metadata** – Description of the data set and its contents
- **Preservation metadata** – Data associated with preserving the integrity of the dataset
- **Administrative metadata** – Constraints
- **Provenance metadata** – Tracing relationships in the data as it moves through a data pipeline; From raw data to a metric or chart, for instance
- **Definitional metadata** – Standardized definitions and vocabularies to aid in documenting and interpreting data and data products

Why Do We Need Metadata?

- A Data System is not software. Software is merely the substrate.
- A Data System is “An organized collection of symbols and processes that may be used to operate on such symbols.”
- A Data System preserves the structure of the real-world system it describes with some loss of information
- This correspondence is what allows us to manipulate data sets to reveal things about the real world:
 - One entity to one table or set of tables
 - One row to one occurrence
 - One column to one construct or data element
- It is the network of relationships that is encoded in our metadata repositories

Why Do We Need Metadata?



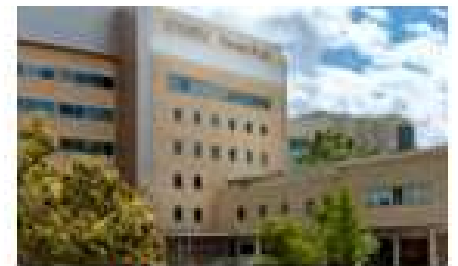
Dispatch



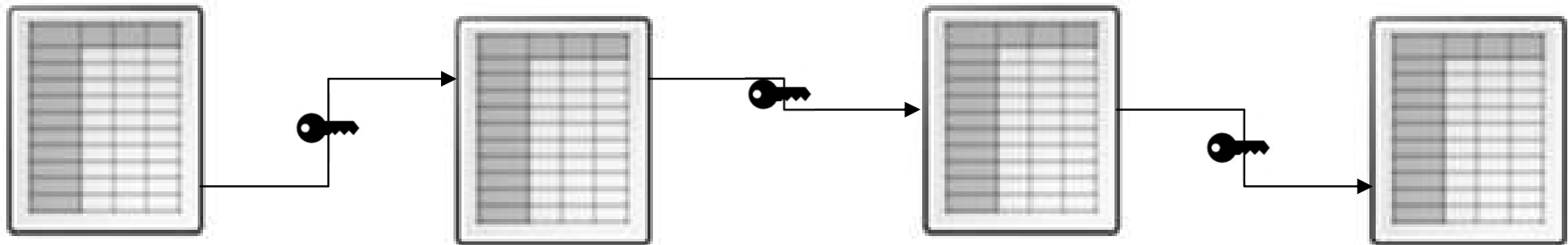
Response



Scene



Destination

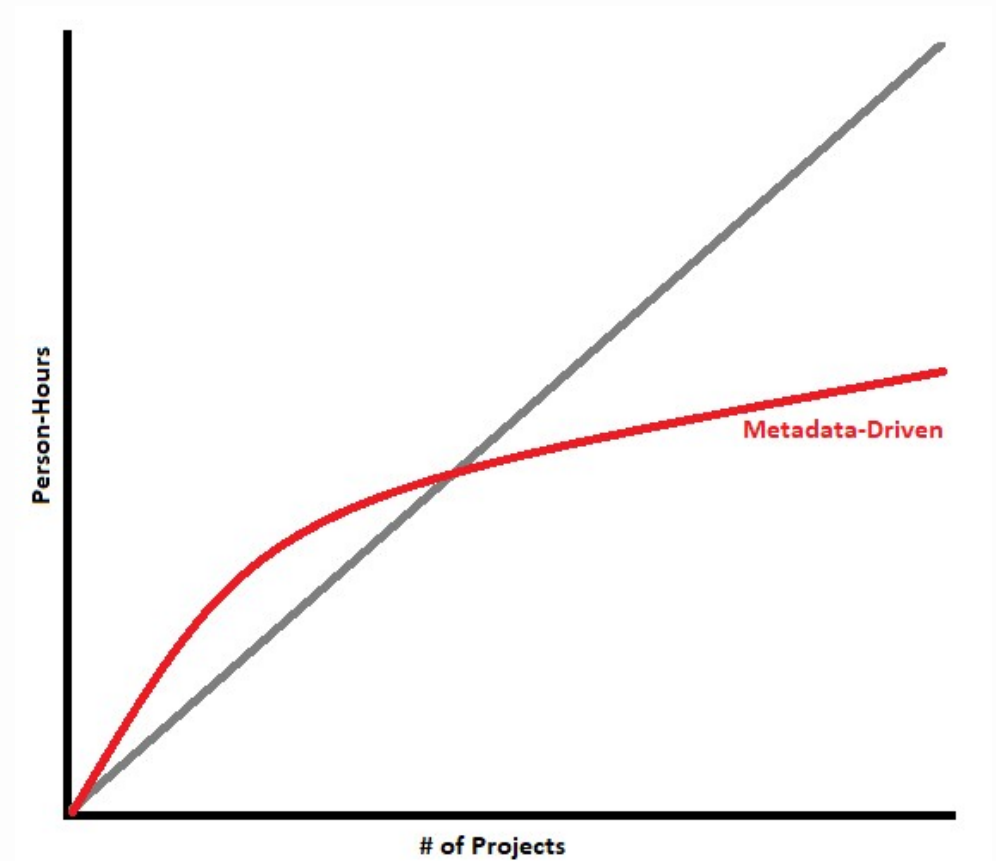


Metadata-Driven Data Science

- We can design our workflows so that we update the metadata repository, and these changes flow downstream to all projects
- Generic functions can be built to take metadata tables as inputs, and use them to extract, analyze and interpret data
- Functions can be updated in one place and one place only when updates/improvements are needed
- This is a preparatory step to full automation of our reporting and dashboard work

Efficiency

- Initial time investment may be greater to set up
- Over time increased efficiency creates an ever-widening gap representing time savings
- Makes current demand for data products sustainable for a small team



Example: Querying Your Database

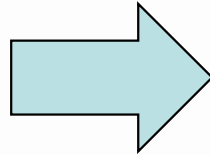
Querying a database is an excellent use case for this kind of workflow

```
123 fact_incident <- EMS_dbobject %>%  
124   tbl(in_schema("DwEms", "Fact_Incident")) %>%  
125   select(., Fact_Incident_PK,  
126         Agency_ID_Internal,  
127         Dim_Agency_FK,  
128         Dim_Incident_Date_FK,  
129         Dim_Incident_FK,  
130         Dim_Patient_FK,  
131         Dim_Disposition_FK,  
132         Patient_Age_In_Years,  
133         Patient_Weight_In_Kilograms) %>%  
134   filter(., Dim_Incident_Date_FK %in% local(dates) &  
135         Agency_ID_Internal %in% local(agency_id_int2)) %>%  
136   collect()
```

If you are hardcoding table and element names for dozens of tables, many projects, and update continually over time...

Example: Querying Your Database

...it is time to build a function:



```

28 extract_db_table <- function(dboobject,
29                               source_schema = NULL,
30                               table_name,
31                               element_list = everything(),
32                               is_date = FALSE,
33                               key_name = NULL,
34                               value_list = NULL){
35
36   ## Construct the tbl statement
37   if (is.null(source_schema) == F) {
38     source_table <- function(input, ...) {
39       tbl(input, in_schema(source_schema, table_name))
40     } else {
41       source_table <- function(input, ...) {
42         tbl(input, table_name)
43       }
44
45   ## Construct the filter statement
46   if (is.null(key_name) == F) {
47     criteria <- function(input, ...) {
48       filter(input, !!rlang::sym(key_name) %in% local(value_list))
49     } else {
50       criteria <- function(input, ...) {
51         filter(input, TRUE)
52       }
53
54   out <- dboobject %>%
55     source_table(.) %>%
56     select(., all_of(element_list)) %>%
57     collect(.) %>%
58     criteria(.)
59
60   out
61 }

```

You might
generalize it
to something
like this...


```

33 ▾ one2one_table_extract <- function(db, dictionary, crosswalk, table_name){
34   ## Pulls the an extract of data elements in the specified table and puts them in order for query
35   table_elements <- dictionary %>% filter(., Table.Name == table_name) %>% arrange(query_order)
36   ## Extracts the name of the schema for elements in the list
37   schema <- table_elements %>% distinct(., Schema) %>% pull()
38   ## Pull the row from the crosswalk containing keys linking fact incident to specified table
39   table_keys <- crosswalk %>%
40     filter(., db_Table2_Name == table_name)
41   ## Pull the key for the source table (in this case Fact_Incident)
42   source_key <- table_keys %>% pull(., Table1_Foreign_Key)
43   ## Pull the table name for the source table (in this case Fact_Incident)
44   source_table_name <- table_keys %>% pull(., db_Table1_Name)
45   ## Pull the key for the specified table being pulled
46   table_key <- table_keys %>% pull(., Table2_Foreign_Key)
47   ## Pull a list of keys from the source table to
48   key_list <- source_table_name %>% get() %>% pull(., source_key)
49   ## Call to extract_db_table that uses the information above to pull specified table from the
50   temp <- extract_db_table(db,
51     source_schema = schema,
52     table_name = table_name,
53     element_list = table_elements %>% pull(Column.Name),
54     key_name = table_key,
55     value_list = key_list) %>%
56     ## Rename data elements using NEMSIS/Standard Element #s
57     set_names(., table_elements %>% pull(Element_Name))
58   ## Join the new 1:1 Table with the analytic dataset
59   analytic_dataset <-< analytic_dataset %>%
60     left_join(., temp,
61       by = setNames(table_key, source_key))
62 ▴ }

```

You can build a wrapper around `extract_db_table` like this!

Extract & Save: The 1:1 Table Function Deconstructed

```
75 ▾ #####
76  ##
77  ##      Extract and save 1:1 Table      #
78  ##      Function deconstructed
79  ##
80 ▾ #####
81
82  db <- Trauma_dbObject
83  dictionary <- trauma_dd
84  crosswalk <- trauma_cw
85  table_name <- "BLOODGAS"
86
```

Extract & Save: The 1:1 Table Function Deconstructed

```
87 ## Pulls the an extract of data elements in the specified table and puts them in order for query
88 table_elements <- dictionary %>% filter(., Table.Name == table_name) %>% arrange(query_order)
```

ID	Element_Name	Element_Number	TraumaOne_Field	Schema	Table.Name	Column.Name	query_order
1	ACCTNO	NA	Trauma Registry Number	NA	BLOODGAS	ACCTNO	0
2	Alcohol screen	NA	Alcohol screen	NA	BLOODGAS	ABG_SRCE	10
3	Alcohol screen results	NA	Alcohol screen results	NA	BLOODGAS	ETOHLEVEL	20
4	Drug use indicator	NA	Drug use indicator	NA	BLOODGAS	ABG_DLVR	30
5	SBIRT indicator	NA	SBIRT indicator	NA	BLOODGAS	ABG_CTRL	40
6	SBIRT result	NA	SBIRT result	NA	BLOODGAS	SBIRT_RES	50
7	SBIRT brief intervention	NA	SBIRT brief intervention	NA	BLOODGAS	SBIRT_INT	60
8	SBIRT referral to treatment	NA	SBIRT referral to treatment	NA	BLOODGAS	SBIRT_REF	70
9	ACCTNO	NA	Trauma Registry Number	NA	CONSULT	ACCTNO	0
10	Consultation date	NA	Consultation date	NA	CONSULT	DT	10
11	Consultation time	NA	Consultation time	NA	CONSULT	TIME	20
12	Consultation service	NA	Consultation service	NA	CONSULT	SPECIALITY	30
13	Consultation physician name	NA	Consultation physician name	NA	CONSULT	CNSLT_DOC	40
14	ACCTNO	NA	Trauma Registry Number	NA	DIAGS	ACCTNO	0
15	Head and neck injury score	NA	Head and neck injury score	NA	DIAGS	AIS_HEAD	10
16	Face injury score	NA	Face injury score	NA	DIAGS	AIS_FACE	20
17	Chest injury score	NA	Chest injury score	NA	DIAGS	AIS_CHEST	30
18	Abdomen and pelvis injury score	NA	Abdomen and pelvis injury score	NA	DIAGS	AIS ABD	40

Extract & Save: The 1:1 Table Function Deconstructed

```
89 ## Extracts the name of the schema for elements in the list
90 if(is.na(table_elements %>% distinct(., schema) %>% pull())){
91   schema <- NULL
92 } else {
93   schema <- table_elements %>% distinct(., schema) %>% pull()
94 }
```

ID	Element_Name	Element_Number	TraumaOne_Field	Schema	Table.Name	Column.Name	query_order
1	ACCTNO	NA	Trauma Registry Number	NA	BLOODGAS	ACCTNO	0
2	Alcohol screen	NA	Alcohol screen	NA	BLOODGAS	ABG_SRCE	10
3	Alcohol screen results	NA	Alcohol screen results	NA	BLOODGAS	ETOHLEVEL	20
4	Drug use indicator	NA	Drug use indicator	NA	BLOODGAS	ABG_DLVR	30
5	SBIRT indicator	NA	SBIRT indicator	NA	BLOODGAS	ABG_CTRL	40
6	SBIRT result	NA	SBIRT result	NA	BLOODGAS	SBIRT_RES	50
7	SBIRT brief intervention	NA	SBIRT brief intervention	NA	BLOODGAS	SBIRT_INT	60
8	SBIRT referral to treatment	NA	SBIRT referral to treatment	NA	BLOODGAS	SBIRT_REF	70

Values	
end_date	"2024-12-31"
schema	NULL
start_date	"2024-01-01"
table_name	"BLOODGAS"

Extract & Save: The 1:1 Table Function Deconstructed

```
95 ## Pull the row from the crosswalk containing keys linking fact incident to specified table
96 table_keys <- crosswalk %>%
97   filter(., db_Table2_Name == table_name)
```

Values	
end_date	"2024-12-31"
schema	NULL
start_date	"2024-01-01"
table_name	"BLOODGAS"

	db_Table1_Name	Table1_Key	Correspondence	db_Table2_Name	Table2_Key	Core	N
1	MAINDATA	ACCTNO	`1:1	BLOODGAS	ACCTNO	0	1
2	CONSULT	ACCTNO	`1:M	MAINDATA	ACCTNO	0	15
3	MAINDATA	ACCTNO	`1:1	DIAGS	ACCTNO	0	1
4	MAINDATA	ACCTNO	`1:1	EMERG	ACCTNO	1	1
5	MAINDATA	ACCTNO	`1:1	FINANCE	ACCTNO	0	1
6	MAINDATA	ACCTNO	`1:1	FLDDETA	ACCTNO	0	1

```
> table_keys
# A tibble: 1 × 7
  db_Table1_Name Table1_Key Correspondence db_Table2_Name Table2_Key Core N
  <chr>          <chr>      <chr>          <chr>          <chr>   <dbl> <dbl>
1 MAINDATA      ACCTNO      `1:1          BLOODGAS      ACCTNO     0     1
```

Extract & Save: The 1:1 Table Function Deconstructed

```
100 ## Pull the table name for the source table
101 source_table_name <- table_keys %>% pull(., db_Table1_Name)
```

	db_Table1_Name	Table1_Key	Correspondence	db_Table2_Name	Table2_Key	Core	N
1	MAINDATA	ACCTNO	1:1	BLOODGAS	ACCTNO	0	1

```
> source_table_name
[1] "MAINDATA"
```

Extract & Save: The 1:1 Table Function Deconstructed

```
98 ## pull the key for the source table
99 source_key <- table_keys %>% pull(., Table1_Key)
```

	db_Table1_Name	Table1_Key	Correspondence	db_Table2_Name	Table2_Key	Core	N
1	MAINDATA	ACCTNO	1:1	BLOODGAS	ACCTNO	0	1

```
> source_key
[1] "ACCTNO"
```

Extract & Save: The 1:1 Table Function Deconstructed

```
102 ## Pull the key for the specified table being pulled
103 table_key <- table_keys %>% pull(., Table2_Key)
```

	db_Table1_Name	Table1_Key	Correspondence	db_Table2_Name	Table2_Key	Core	N
1	MAINDATA	ACCTNO	1:1	BLOODGAS	ACCTNO	0	1

```
> table_key
[1] "ACCTNO"
```


Extract & Save: The 1:1 Table Function Deconstructed

```
104 ## Pull a list of keys from the source table to
105 key_list <- source_table_name %>% get() %>% pull(., source_key)
```

```
107 ## Pull a list of keys from the source table to
108 key_list <- MAINDATA %>% get() %>% pull(., ACCTNO)
```

	ACCTNO	Abstractor	Entry into trauma system	Hospital medical record number	State Trauma Band number
1	27991				
2	29731				
3	80588				
4	113448				
5	117791				
6	131235				
7	135836				
8	140034				
9	140077				
10	140094				
11	140106				
12	140107				
13	140109				
14	140117				
15	140123				
16	140131				
17	140132				
18	140133				

Values	
end_date	"2024-12-31"
key_list	int [1:23311] 27991 29731 80588 113448 117791
schema	NULL
source_key	"ACCTNO"
source_table_name	"MAINDATA"
start_date	"2024-01-01"
table_key	"ACCTNO"
table_name	"BLOODGAS"

Extract & Save: The 1:1 Table Function Deconstructed

```

111 ## Call to extract_db_table that uses the information above to pull specified table from the db
112 temp <- extract_db_table(db,
113                           source_schema = schema,
114                           table_name = table_name,
115                           element_list = table_elements %>% pull(column_name),
116                           key_name = table_key,
117                           value_list = key_list) %>%
118 
```

Trauma_dbobject Formal class Microsoft SQL Server

ID	Element_Name	Element_Number	TraumaOne_Field	Schema	Table_Name	Column_Name	query_order
1	ACCTNO	NA	Trauma Registry Number	NA	BLOODGAS	ACCTNO	0
2	Alcohol screen	NA	Alcohol screen	NA	BLOODGAS	ABG_SRCE	10
3	Alcohol screen results	NA	Alcohol screen results	NA	BLOODGAS	ETOHLEVEL	20
4	Drug use indicator	NA	Drug use indicator	NA	BLOODGAS	ABG_DLVR	30
5	SBIRT indicator	NA	SBIRT indicator	NA	BLOODGAS	ABG_CTRL	40
6	SBIRT result	NA	SBIRT result	NA	BLOODGAS	SBIRT_RES	50
7	SBIRT brief intervention	NA	SBIRT brief intervention	NA	BLOODGAS	SBIRT_INT	60
8	SBIRT referral to treatment	NA	SBIRT referral to treatment	NA	BLOODGAS	SBIRT_REF	70

```

values
end_date      "2024-12-31"
schema        NULL
start_date    "2024-01-01"
table_name    "BLOODGAS"

```

```

extract_db_table(Trauma_dbobject,
                  source_schema = NULL,
                  table_name = "BLOODGAS",
                  element_list = all_of("ACCTNO", "ABG_SRCE",
                                         "ETOHLEVEL", "ABG_DLVR",
                                         "ABG_CTRL", "SBIRT_RES",
                                         "SBIRT_INT", "SBIRT_REF"),
                  key_name = "ACCTNO",
                  value_list = c(27991, 29731, 80588, 113448,
                                117791, 131235, ...))

```

```

> table_key
[1] "ACCTNO"

```

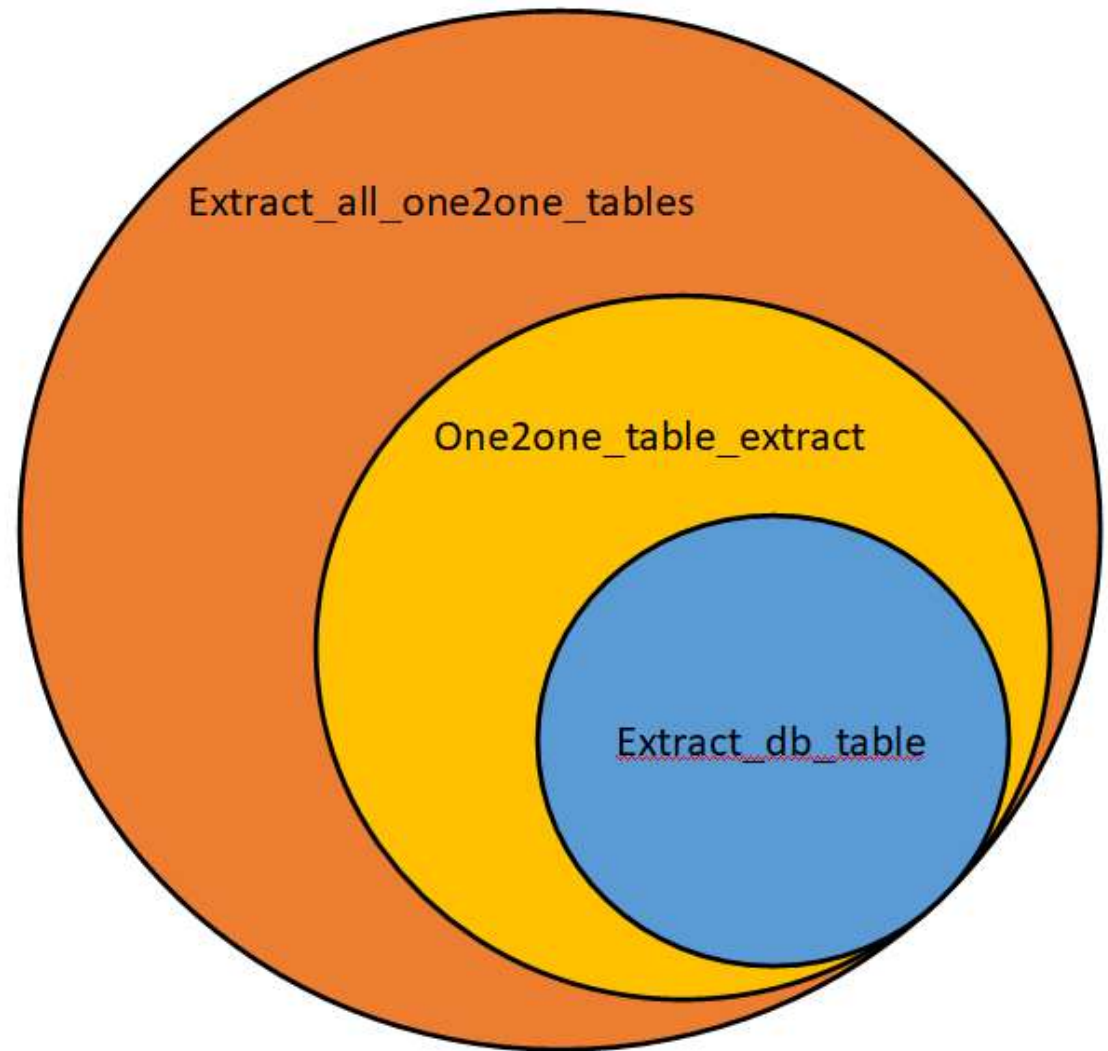
```

values
end_date      "2024-12-31"
key_list       int [1:23311] 27991 29731 80588 113448 117791
schema        NULL
source_key     "ACCTNO"
source_table_name "MAINDATA"
start_date    "2024-01-01"
table_key     "ACCTNO"
table_name    "BLOODGAS"

```

Takeaways: The 1:1 Table Function

- Functions are nested
- At the outermost circle fewest arguments:
 - db
 - dictionary
 - crosswalk
- All other arguments are created within the data pipeline as you move deeper



Database Agnostic

- These functions can be used with our Oregon Trauma Registry (OTR) data or the Oregon EMS Information System (OR-EMSIS)
- These two databases have very different structures
- The basic functions are agnostic to the database they operate on
- All it requires is a properly formatted set of data dictionary tables

Metadata-Driven Data Science

- Other metadata driven workflows are possible:
 - Workflows to update and modify the metadata repositories
 - Workflows to extract and update documentation of value lists, constraints and regex patterns to support data validation
 - Functions to support automated update and deduplication of data sets
 - Data structures to store case definitions and denominator inclusion for calculation of metrics (and beyond)
 - Dataset picker apps
 - Cross-dataset mappings for metadata driven approaches to automating data linkage

Conclusion: Getting upstream

- The metadata driven approach to projects supports:
 - Use of generic scripts for diverse tasks
 - Modular code development
- It has the potential to increase capacity of small teams
- Offers a low resource path out of the public sector quandary



Questions?

Contact:
Peter Geissert@oha.oregon.gov



(Enter) DEPARTMENT (ALL CAPS)
(Enter) Division or Office (Mixed Case)
